Mini-projeto de PPP 2022/2023

No âmbito da cadeira de Princípios de Programação Procedimental foi-nos proposto a realização de um projeto que consiste no desenvolvimento de uma aplicação para auxiliar um funcionário de uma oficina na gestão das suas reservas. Para isso, utilizamos a aplicação VSCode (Visual Studio Code) para executar o nosso projeto. Assim criamos 3 ficheiros com código fonte, o projeto.c que corresponde ao main com as opções de execução, o projeto_struct.h correspondente ao conjunto de structs que vão ser utilizadas e por fim o projeto_funcoes.c que corresponde a todas as funções que utilizamos ao longo do projeto.

FICHEIRO PROJETO.C

Esta parte do projeto é onde se localiza o main que vai orientar o utilizador na escolha das opções que deseja utilizar.

- Primeiramente são incluídos os cabeçalhos necessários e a declaração das estruturas e funções utilizadas no programa.
- De seguida temos a função main onde são criadas três listas vazias: listaClientes, listaReservas e listaPreReservas.
- Temos a função verifica_ficheiros que passa as três listas como argumento, esta verifica se os ficheiros de dados correspondentes a cada uma das listas estão vazios, e se não estiverem, carrega os dados para as respetivas listas.
- Em seguida, o loop while é iniciado (com a variável opção inicializada a zero) e só termina quando a opção selecionada for 12(“Sair”).
- É utilizado o switch-case para que em cada caso seja chamada uma função especifica e executado o seu código.
- O menu é exibido ao utilizador com várias opções numerada de 1 a 12, facilitando o usuário a escolher a opção pretendida. Assim que o número desejado seja escolhido a sua funcionalidade é executada e o programa retorna ao menu para permitir que o usuário escolha outra opção.

FICHEIRO PROJETO_STRUCT.H

Este ficheiro é utilizado principalmente como cabeçalho, neste estão declaradas estruturas de dados, constantes e os nomes das funções que são utilizadas no projeto. Assim, permite que outros ficheiros fonte incluam este cabeçalho e utilizem as definições e funções nele contidos sem precisar de as escrever novamente.

- Neste caso, primeiramente tem a inclusão das bibliotecas padrão permitindo que as estruturas e operações definidas nesse ficheiro utilizem as funcionalidades já implementadas nessas bibliotecas.

Em seguida são definidas diversas estruturas tais como:

- "Data" que possui os campos 'dia', 'mês' e 'ano' que representam uma data.
- "Hora" que possui os campos 'hora, 'minuto' que representam uma hora.
- "Cliente" que possui os campos 'nome', 'apelido, 'nif'. O campo 'nome' armazena o nome do cliente, o campo 'apelido' armazena o apelido e o campo 'nif' armazena o NIF do cliente.
- "noLista" é utilizada para criar nós em uma lista, onde cada nó contém um campo chamado 'cliente' do tipo 'Cliente', que armazena as informações do cliente. Cada nó possui um campo chamada 'prox' que é um ponteiro para o próximo nó da lista. No fim é criado o 'pLista' que é um ponteiro para um nó da lista (do tipo noLista).
- "Marcacao" é utilizada para armazenar os dados de uma marcação de serviço (reserva ou pré-reserva). Possui o campo 'servico' do tipo inteiro que indica se é lavagem (0) ou manutenção (1); o campo 'horas' que é do tipo "Hora"; o campo 'data' que é do tipo "Data"; e o campo 'cliente' que é do tipo "pLista" onde este é um ponteiro para a lista de clientes.
- "noListaMarcacao" é utilizada para criar nós em uma lista, onde cada nó contém um campo chamado 'pessoareserva' que é do tipo "Marcacao" e que armazena as informações da marcação. Cada nó possui um campo chamada 'prox' que é um ponteiro para o próximo nó da lista. No fim é criado o 'pListaMarcacao' que é um ponteiro para um nó da lista (do tipo noListaMarcacao).

FICHEIRO PROJETO_STRUCT.C

- **pLista cria_listaClientes ()**
  - Esta função cria um nó vazio e retorna um ponteiro para esse nó, que será usado como o início da lista de clientes.

- **int vazia_listaClientes (pLista lista)**
  - Esta função verifica de a lista de clientes está vazia, verificando se o campo 'prox' do primeiro nó é NULL, se não estiver a NULL é porque não está vazia retornando 0.

- **pLista destroi_listaClientes (pLista lista)**
- **pListaMarcacao destroi_marcacao (pListaMarcacao lista)**

  - Esta função percorre a lista de clientes, libertando a memoria de cada nó até esta estar vazia, retornado um ponteiro nulo apontando que a lista foi completamente destruída.

- **void procura_listaClientes (pLista lista, int chave, pLista *ant, pLista *actual)**

  - Nesta função existem os ponteiros 'ant' e 'actual', do tipo pLista, em que o 'ant' aponta para o início da lista e o 'actual' aponta para o próximo nó da lista. É inicializado um loop que enquanto o valor 'actual' não for NULL e o valor do nif do cliente apontado pelo 'actual' for menor que o valor da chave, vai atualizar para onde os ponteiros devem apontar de forma adequada. Quando chegou ao final da lista, sai do loop e vai verificar se o nif do cliente apontado pelo 'actual' é diferente á chave, caso seja o ponteiro 'actual' é definido como nulo.

- **void elimina_listaClientes (pLista lista, int chave)**

  - Nesta função é chamada função 'procura_listaClientes' para encontrar o nó correspondente á chave na lista. Se o 'actual' não for NULL significa que encontrou a chave e assim pode proceder á eliminação do nó. Para o eliminar o ponteiro 'prox' do nó anterior é atualizado para apontar para o nó seguinte ao nó atual, removendo assim o nó atual e libertando o espaço de memória(free()).

- **void insere_cliente (pLista lista, Cliente p1);**

    - A função começa por criar um novo nó (‘no’) usando a função malloc para alocar memória para o nó, se a alocação de memoria for bem-sucedida (diferente de NULL) é atribuído ao campo cliente do nó os dado do cliente (‘p1’), em seguida é chamada a função ‘procura_listaClientes’ para encontrar a posição onde colocar o cliente na lista com base no seu NIF. Após encontrada a posição correta os ponteiros são atualizados para inserir o novo nó na lista de clientes.

- **pLista pesquisa_clientes (pLista lista, int chave);**

    - Nesta função é chamada a função ‘procura_listaClientes’ para pesquisar a chave, e caso seja encontrada vai retornar o seu valor.

- **void imprime_listaClientes (pLista lista);**

    - Nesta função é declarado um ponteiro aux que é do tipo pLista que só começa no segundo nó para que este ignore o nó de cabeçalho. É inicializado um loop que enquanto o aux não for NULL dá print aos dados do nó da lista e no fim passa para o próximo nó.

- **int existe_nif (pLista lista, int nif)**

    - Esta função recebe a lista de clientes e uma variável nif e começa por criar uma variável ‘aux’ do tipo pLista que é um ponteiro inicializado para o início da lista. Percorre a lista e se encontrar um nó que contenha o mesmo nif que a variável nif vai retornar 1, caso contrário retorna 0.

- **void menu_add_cliente (pLista lista)**

    - É criado um novo nó (aux) por meio da função calloc, este só será utilizado para armazenar temporariamente as informações do cliente antes de inseri-lo na lista. Em seguida são pedidas as informações necessárias ao cliente. É inicializado um loop infinito para validar o NIF do usuário, primeiramente é verificado se só existem números inteiros no NIF, se só forem números inteiros tem que ser verificado se o NIF contem 9 dígitos e para isso é criado um loop para ir dividindo o NIF por 10, se o numero de dígitos forem diferente de 9 aparece a mensagem de NIF incorreto ao utilizador, caso contrário vai chamar a função “exite_nif” e se esta retornar 1 é porque o NIF já existe, mas caso retorne 0 a função “insere_cliente” é chamada e vai inserir os dados todos do cliente na lista.

- **void mostra_lista_clientes (pLista lista)**

- Esta função chama o imprime_listaClientes exibindo a lista dos clientes no terminal.

- **pListaMarcacao cria_marcacao ()**
  - Esta função cria um ponteiro para um novo nó da lista alocando-o dinamicamente utilizando malloc de modo a ser utilizado o tamanho adequado. O campo prox do nó é definido sendo NULL. Retornando no final o ponteiro para o novo nó.

- **void procura_marcacao (pListaMarcacao listar, int chave,int chave2, int chave3, int chave4, int chave5, pListaMarcacao *ant, pListaMarcacao *actual);**

  - Esta função recebe a lista de reservas ou pre-reservas, recebe ainda 5 variáveis que vão ser os critérios do if para podermos ordenar a lista (as mais antigas primeiro). Definimos dois ponteiros do tipo pListaMarcacao o "ant" e o "actual", que serão usados para retornar o nó anterior e o nó atual na lista de marcações.
  - Definimos o ponteiro ant como apontando para o início da lista e o ponteiro actual apontando para o proximo nó da lista.
  - O ciclo while vai ser executado enquanto *actual for diferente de NULL (ou seja, ainda existem nós na lista). Cada critério separado por operadores "&&" ou "||" verifica se os valores dos campos ano, mes, dia, hora e minuto da estrutura pessoareserva do nó atual são maiores do que as chaves correspondentes fornecidas. Se forem maiores os ponteiros ant e actual são atualizados para avançar para o próximo nó na lista.
  - Se o ciclo while terminar e não tiver encontrado nehuma correspondência o ponteiro actual será definido como NULL, o que indica que a marcação não foi encontrada na lista

- **void procura_marcacao_do_insere_marcacoes_cliente (pListaMarcacao listar, int chave, int chave2, int chave3, int chave4, int chave5, pListaMarcacao *ant, pListaMarcacao *actual);**

  - Esta função recebe as listas auxiliares de reservas ou pre-reservas, recebe ainda 5 variáveis que vão ser os critérios do 'if' para podermos ordenar a lista (as mais recentes primeiro). Definimos dois ponteiros do tipo pListaMarcacao o "ant" e o "actual", que serão usados para retornar o nó anterior e o nó atual na lista de marcações.
  - Definimos o ponteiro ant como apontandor para o início da lista e o ponteiro actual apontando para o proximo nó da lista.
  - O ciclo while vai ser executado enquanto *actual for diferente de NULL (ou seja, ainda existem nós na lista). Cada critério separado por operadores "&&" ou "||" verifica se os valores dos campos ano, mes, dia, hora e

minuto da estrutura pessoareserva do nó atual são maiores do que as chaves correspondentes fornecidas. Se forem maiores os ponteiros ant e actual são atualizados para avançar para o próximo nó na lista.

- Se o ciclo while terminar e não tiver encontrado nehuma correspondência o ponteiro actual será definido como NULL, o que indica que a marcação não foi encontrada na lista.

- **void elimina_marcacao (pListaMarcacao listar, int chave, int chave2, int chave3, int chave4, int chave5)**

  - Nesta função declaramos os ponteiros 'ant' e 'actual' que iriamos precisar ao longo da função
  - É chamada função 'procura_marcacao' para encontrar o nó correspondente á chave na lista. Se o 'actual' não for NULL significa que encontrou a chave e assim pode proceder á eliminação do nó. Para o eliminar o ponteiro 'prox' do nó anterior é atualizado para apontar para o nó seguinte ao nó atual, removendo assim o nó atual e libertando o espaço de memória(free()).

- **void insere_marcacao (pListaMarcacao listar, Marcacao p1);**

  - A função começa por criar um novo nó ('no') usando a função malloc para alocar memória para o nó, se a alocação de memoria for bem-sucedida (diferente de NULL) é atribuído ao campo pessoareserva do nó os dados da marcação ('p1'), em seguida é chamada a função 'procura_marcacao' para encontrar a posição onde colocar o cliente na lista com base na sua data (as mais antigas primeiro). Após encontrada a posição correta os ponteiros são atualizados para inserir o novo nó na lista de marcações.

- **void insere_marcacoes_cliente (pListaMarcacao listar, Marcacao p1);**

  - A função começa por criar um novo nó ('no') usando a função malloc para alocar memória para o nó, se a alocação de memória for bem-sucedida (diferente de NULL) é atribuído ao campo pessoareserva do nó os dados da marcação ('p1'), em seguida é chamada a função 'procura_marcacao' para encontrar a posição onde colocar o cliente na lista com base na sua data (as mais recentes primeiro). Após encontrada a posição correta os ponteiros são atualizados para inserir o novo nó na lista de marcações.

- **pListaMarcacao pesquisa_marcacao (pListaMarcacao listar, int chave, int chave2, int chave3, int chave4, int chave5);**
  - Nesta função é chamada a função 'procura_marcacao' para pesquisar a chave, e caso seja encontrada vai retornar o seu valor.

- **void imprime_marcacao (pListaMarcacao listar);**

    - Nesta função é declarado um ponteiro aux que é do tipo pListaMarcacao que só começa no segundo nó para que este ignore o nó de cabeçalho. É inicializado um loop que enquanto o aux não for NULL dá print aos dados do nó da lista e no fim passa para o próximo nó.

- **int existe (pListaMarcacao listar,int servico, int minuto, int hora, int dia, int mes, int ano, int hora_aux, int minuto_aux, int hora_aux2, int minuto_aux2);**

    - Nesta função é declarado um ponteiro atual_aux que é do tipo pListaMarcacao.

    - É inicializado um loop que enquanto o atual_aux não for NULL vai verificar 3 condições: se existe uma reserva na meia hora anterior e se é uma reserva que dure 1 hora(manutenção), se a reserva que estamos a tentar inserir for uma manutenção (1 hora) verifica a meia hora a seguir para saber se já existe lá alguma reserva, se existe alguma reserva à hora que estamos a tentar inserir a reserva.Cada condição retorna 1 se se verificar.Se percorrer a lista de reservas e nenhuma destas condições se verificar a função retorna 0.

- **void menu_add_marcacao (pListaMarcacao listar, pLista lista, pListaMarcacao listaPreReservas);**

    - É criado um novo nó (aux) por meio da função calloc, este só será utilizado para armazenar temporariamente as informações do cliente antes de inseri-lo na lista. Em seguida são pedidas as informações necessárias ao cliente, e para os scanf usados foram feitas as devidas proteções.
    - Criamos as variáveis a "minutos_auxiliar" e "hora_auxiliar" que toma o valor da meia hora anterior á reserva

- **void mostra_lista_marcacao (pListaMarcacao listar)**

    - Esta função chama o imprime_listaClientes exibindo a lista dos clientes no terminal.

- **void cancelar_reserva (pListaMarcacao listar, pListaMarcacao listaPreReservas)**

    - Função do tipo void usada para cancelar uma reserva que recebe 2 parametros: pListaMarcacao listar (Lista de reservas) e pListaMarcacao listaPreReservas (Lista de Pre-Reservas). Nesta função começámos por definir variáveis dos tipos int e declaramos varios ponteiros que iriamos precisar ao longo da

função. Inicializámos as variáveis “verifica”, “verifica_auxiliar”, “vazia”, “vazia2” e “vazia3” a 0 e pedimos alguns dados da reserva a cancelar ao utilizador e associámos os valores às suas respetivas variáveis.

- Utilizámos um ciclo while para percorrer uma lista de reservas e realizar várias verificações e ações com base em diferentes condições. O código verifica se uma reserva existe na lista de acordo com os dados pedidos ao utilizador e, em seguida, verifica se a meia hora seguinte à reserva está ocupada. Dependendo do valor da variável "serviço", o código executa diferentes ações.
- Se o valor de "serviço" for 1(manutenção), o código cancelou uma reserva de 1 hora e procura por pré-reservas do tipo manutenção para preencher o tempo disponível na lista de reservas. Se não houver pré-reservas do tipo manutenção(1 hora), procura-se por pré-reservas do tipo lavagem(30 minutos). O código continua a verificar se existem pré-reservas disponíveis para preencher o tempo livre restante.
- Se o valor de "serviço" for 0(lavagem), o código cancelou uma reserva de 30 minutos e verifica se a meia hora seguinte está vazia. Se estiver vazia, procuramos por pré-reservas do tipo manutenção para preencher o tempo disponível. Se nenhuma pré-reserva for encontrada para preencher o tempo disponível, o código verifica se a hora anterior à reserva cancelada está disponível. Se estiver, procura-se por uma pré-reserva do tipo manutenção que comece na meia hora anterior para inserir nesse tempo senão procura apenas uma pre-reserva de 30 minutos para inserir nos 30 minutos disponíveis
- Dentro de cada ciclo while que usámos para verificar se existiam pré-reservas na lista implementámos um ciclo while, para verificar se existem pré-reservas com a mesma data e hora igual e se existirem ele vai mudar a pré-reserva que tinha encontrado para essa até a pré-reserva a seguir a essa ser diferente, pois como a lista está ordenada das mais recentes para as mais antigas nós queremos inserir a pré-reserva que está á mais tempo em lista de espera.
- O código utiliza ciclos while para percorrer a lista de pré-reservas e encontrar as pré-reservas adequadas para inserir na lista de reservas. Também é usado um if para remover da lista de pré-reservas o nó onde estava a pré-reserva após ser inserida na lista de reservas.

- **void cancelar_pre_reserva (pListaMarcacao listaPreReservas)**

  - Função do tipo void que recebe o parâmetro pListaMarcacao listaPreReservas(Lista das Pré-Reservas).Nesta função começámos por definir variáveis dos tipos int e declaramos varios ponteiros que iriamos precisar ao longo da função e pedimos os dados da pré-reserva a cancelar ao utilizador associando os valores às suas respetivas variáveis. Implementámos um ciclo while para procurar a pré-reserva a ser cancelada na lista de pré-reservas e se for encontrada removemos o nó correspondente a essa pré-reserva. Se não for encontrada nenhuma pré-

reserva de acordo com os dados inseridos recorremos a um “printf()” para imprimir que não foi possível encontrar a pré-reserva.

- **void cancelar_realizar_servico (pListaMarcacao lista)**

    - Função do tipo void que recebe o parâmetro pListaMarcacao lista (Lista das Reservas). Nesta função começámos por definir variáveis dos tipos int e declaramos varios ponteiros que iriamos precisar ao longo da função e pedimos os dados da reserva que vai ser executada ao utilizador associando os valores às suas respetivas variáveis. Implementámos um ciclo while para procurar a reserva na lista e se for encontrada removemos o nó correspondente a essa reserva. Se não for encontrada nenhuma pré-reserva de acordo com os dados inseridos recorremos a um “printf()” para imprimir que não foi possível encontrar a reserva.

- **void listar_marcacao_cliente (pListaMarcacao listar, pListaMarcacao listaPreReservas)**

    - Função do tipo void que recebe os parâmetros pListaMarcacao listar (Lista das Reservas) e pListaMarcacao listaPreReservas (Lista das Pre-Reservas). Definimos uma variavel nif do tipo int que vai ter o valor que vamos pedir ao utilizador.
    - Criamos 2 listas auxiliares uma para reservas e outra para pré-reservas e verificamos percorrendo as listas recebidas nos parâmetros se existem reservas e pré-reservas para o nif inserido e se existirem vamos inseri-las nas listas auxiliares correspondentes e vamos imprimir as duas listas auxiliares recorrendo a dois printf().

- **void escreve_para_ficheiro_clientes (pLista lista)**
- **void escreve_para_ficheiro_reservas (pListaMarcacao listar)**
- **void escreve_para_ficheiro_prereservas (pListaMarcacao listaPreReservas)**

    - Funções do tipo void que recebem os parâmetros pLista lista (lista dos clientes), pListaMarcacao listar (lista das reservas), pLista listaMarcacao (lista das pré-reservas). Definimos variáveis importantes para a implementação das funções.
    - As funções são responsáveis por escrever dados num ficheiro dependendo da função.
    - Começámos por abrir o arquivo ("clientes.txt" ou “reservas.txt” ou “pre-reservas.txt” dependendo da função) no modo escrita “w” usando a função fopen

- Usámos um ciclo while que percorre a lista respetiva de cada função para escrevermos os seus dados no ficheiro recorrendo á função fprintf. No fim da função o ficheiro é fechado usando a função fclose.

- **void carrega_de_ficheiro_clientes (pLista lista)**
- **void carrega_de_ficheiro_reservas (pListaMarcacao listar)**
- **void carrega_de_ficheiro_prereservas (pListaMarcacao listaPreReservas)**

  - Funções do tipo void que recebem os parâmetros pLista lista (lista dos clientes), pListaMarcacao listar (lista das reservas), pLista listaMarcacao (lista das pré-reservas). Definimos variáveis importantes para a implementação das funções.
  - As funções são responsáveis por ler dados de um ficheiro e carregar esses dados para a lista correspondente.
  - Começámos por abrir o ficheiro ("clientes.txt" ou “reservas.txt” ou “pre-reservas.txt”) no modo de leitura “r” usando a função fopen, usámos um ciclo while com a função fscanf para ler os dados do ficheiro. Dentro do ciclo while alocamos a memória para uma nova estrutura Cliente usando a função malloc. Os dados lidos do ficheiro são copiados para a estrutura alocada usando a função strcpy e chamamos a função “insere_cliente_do_ficheiro” tendo como argumentos a lista onde vai inserir e a estrutura. No final das funções fechamos o ficheiro usando a função fclose.

- **int verifica_ficheiros (pLista lista, pListaMarcacao listar, pListaMarcacao listaPreReservas)**

  - Esta função começa por abrir os 3 ficheiros em modo de leitura, e para cada um dos ficheiros vai verificar se foi aberto com sucesso. Se foi aberto com sucesso utiliza o fseek para mover o cursor para o final do ficheiro e a função ftell para obter a posição atual onde esta representa o tamanho do ficheiro, em seguida fecha o ficheiro. Se o tamanho for 0 é porque não existem dados, caso contrário chama a função “carrega_de_ficheiro_clientes()” que vai carregar as informações do ficheiro para a lista correspondente.